

Exercícios autorais

Preliminares

Conteúdo autoral derivado do lesson model aprovado; a bibliografia é complementar.

1. Razões para estudar conceitos de LP

Caso prático: uma equipe precisa escolher uma linguagem para um novo sistema. Mostre como “Razões para estudar conceitos de LP” deve entrar na análise e indique uma consequência concreta da decisão.

A equipe deve considerar os conceitos de LP para escolher uma linguagem adequada ao problema, em vez de apenas utilizar aquela que já conhece. Uma consequência concreta é obter uma solução mais adequada, evitando recursos improvisados, difíceis ou menos seguros.

2. Domínios de Programação

Procure o erro: alguém afirmou que não é preciso considerar “Domínios de Programação” quando o código já funciona. Identifique o problema desse raciocínio e reescreva a conclusão.

O erro é pensar que o simples funcionamento do código garante que a linguagem seja adequada. Diferentes domínios possuem necessidades diferentes, portanto a conclusão correta é que o domínio da aplicação deve ser considerado na escolha da linguagem.

3. Critérios de Avaliação de Linguagem

Sugira uma abordagem para avaliar “Critérios de Avaliação de Linguagem” durante uma revisão técnica. Apresente o critério usado e uma evidência que sustentaria sua recomendação.

Durante a revisão, deve-se avaliar a linguagem conforme critérios como legibilidade, facilidade de escrita e confiabilidade. Uma evidência é verificar se o código é fácil de entender e manter, pois a legibilidade influencia diretamente a manutenção.

4. Legibilidade

Compare duas alternativas hipotéticas sob a perspectiva de “Legibilidade”. Indique em que cenário cada alternativa faria mais sentido.

Entre uma linguagem simples e adequada ao domínio e outra mais complexa e inadequada, a primeira faz mais sentido quando a prioridade é facilitar leitura e manutenção. A segunda só seria justificável quando oferecer recursos necessários ao domínio específico.

5. Ortogonalidade

Você precisa defender uma decisão profissional relacionada a “Ortogonalidade”. Estruture uma justificativa curta com contexto, critério e trade-off.

Em um sistema que prioriza simplicidade e facilidade de manutenção, é vantajoso escolher uma linguagem com boa ortogonalidade, pois ela reduz exceções e facilita o aprendizado. O trade-off é que ortogonalidade excessiva pode gerar combinações complexas e desnecessárias.

6. Capacidade de escrita

Caso prático: uma equipe precisa escolher uma linguagem para um novo sistema. Mostre como “Capacidade de escrita” deve entrar na análise e indique uma consequência concreta da decisão.

A equipe deve avaliar o quanto a linguagem facilita a criação de programas para o domínio do sistema. Uma consequência concreta é que uma linguagem adequada ao problema pode reduzir o esforço de desenvolvimento, enquanto uma inadequada pode exigir soluções mais complexas.

7. Suporte para abstração

Procure o erro: alguém afirmou que não é preciso considerar “Suporte para abstração” quando o código já funciona. Identifique o problema desse raciocínio e reescreva a conclusão.

O erro é considerar apenas se o código funciona, ignorando a facilidade de representar soluções complexas. A conclusão correta é que o suporte à abstração também deve ser considerado, pois recursos adequados permitem desenvolver soluções mais organizadas e apropriadas ao problema.

8. Expressividade

Sugira uma abordagem para avaliar “Expressividade” durante uma revisão técnica. Apresente o critério usado e uma evidência que sustentaria sua recomendação.

Deve-se verificar se a linguagem permite representar as operações de forma conveniente e concisa. Uma evidência é comparar a quantidade e a complexidade de código necessária para realizar a mesma tarefa; construções como `count++` demonstram maior expressividade.

9. Confiabilidade

Compare duas alternativas hipotéticas sob a perspectiva de “Confiabilidade”. Indique em que cenário cada alternativa faria mais sentido.

Uma linguagem com boa verificação de tipos e tratamento de exceções faz mais sentido em sistemas críticos, enquanto uma alternativa com menos mecanismos de segurança pode ser aceitável em sistemas menos críticos. A escolha deve considerar o risco e o custo de possíveis falhas.

10. Verificação de tipos

Você precisa defender uma decisão profissional relacionada a “Verificação de tipos”. Estruture uma justificativa curta com contexto, critério e trade-off.

Em um sistema que exige alta confiabilidade, deve-se preferir uma linguagem que faça verificação de tipos, principalmente em tempo de compilação. O trade-off é que verificações adicionais podem aumentar custos ou restrições, mas ajudam a detectar erros mais cedo e reduzir problemas em execução.

11. Manipulação de exceções

Caso prático: uma equipe precisa escolher uma linguagem para um novo sistema. Mostre como “Manipulação de exceções” deve entrar na análise e indique uma consequência concreta da decisão.

A equipe deve considerar se a linguagem possui mecanismos adequados para tratar erros durante a execução. Uma consequência concreta é que uma linguagem com bom tratamento de exceções permite interceptar erros, tomar medidas corretivas e aumentar a confiabilidade do sistema.

12. Aliasing (apelidos)

Procure o erro: alguém afirmou que não é preciso considerar “Aliasing (apelidos)” quando o código já funciona. Identifique o problema desse raciocínio e reescreva a conclusão.

O erro é ignorar os apelidos apenas porque o programa funciona atualmente. Como diferentes nomes podem acessar a mesma memória, uma alteração por um deles pode afetar o outro. Portanto, o uso de apelidos deve ser considerado por seu potencial de reduzir a confiabilidade.

13. Custo

Sugira uma abordagem para avaliar “Custo” durante uma revisão técnica. Apresente o critério usado e uma evidência que sustentaria sua recomendação.

A avaliação deve considerar o custo total, incluindo treinamento, desenvolvimento, compilação, execução, implementação, falhas e manutenção. Uma evidência importante é que a manutenção pode representar até duas a quatro vezes o custo de desenvolvimento em sistemas grandes e de longa duração.

14. Portabilidade e generalidade

Compare duas alternativas hipotéticas sob a perspectiva de “Portabilidade e generalidade”. Indique em que cenário cada alternativa faria mais sentido.

Uma linguagem mais padronizada e portátil faz mais sentido quando o sistema precisa ser executado em diferentes plataformas. Já uma linguagem mais especializada pode ser melhor quando o objetivo é atender eficientemente a um domínio específico, mesmo com menor generalidade ou portabilidade.

15. Influências no Projeto de Linguagens

Você precisa defender uma decisão profissional relacionada a “Influências no Projeto de Linguagens”. Estruture uma justificativa curta com contexto, critério e trade-off.

A decisão deve considerar as necessidades do projeto e os fatores que influenciam o projeto da linguagem, como facilidade de escrita, legibilidade, confiabilidade e custo. O critério principal deve ser a adequação da linguagem ao problema. O trade-off é que uma linguagem pode oferecer melhores recursos para determinado domínio, mas apresentar maior complexidade ou custo de aprendizagem.

16. Arquitetura de von Neumann

Caso prático: uma equipe precisa escolher uma linguagem para um novo sistema. Mostre como “Arquitetura de von Neumann” deve entrar na análise e indique uma consequência concreta da decisão.

A arquitetura de von Neumann deve ser considerada porque a forma como o computador organiza memória, processamento e execução influencia a maneira como os programas são executados. Assim, a equipe deve avaliar como a linguagem e sua implementação utilizam esses recursos. Uma consequência concreta é que uma escolha mais adequada pode resultar em melhor eficiência na execução do sistema.

17. Ciclo de busca-decodifica-executa

Procure o erro: alguém afirmou que não é preciso considerar “Ciclo de busca-decodifica-executa” quando o código já funciona. Identifique o problema desse raciocínio e reescreva a conclusão.

O erro está em considerar que o funcionamento do código elimina a necessidade de compreender como ele é executado. O ciclo de busca-decodifica-executa ajuda a entender como as instruções são processadas pelo computador e pode explicar questões de desempenho e implementação. Portanto, mesmo que o código funcione, esse ciclo pode ser relevante para analisar seu comportamento e eficiência.

18. Metodologias de Projeto de Software

Sugira uma abordagem para avaliar “Metodologias de Projeto de Software” durante uma revisão técnica. Apresente o critério usado e uma evidência que sustentaria sua recomendação.

Durante a revisão técnica, deve-se avaliar se a metodologia utilizada é adequada às características do projeto e facilita seu desenvolvimento e manutenção. A recomendação deve ser sustentada por evidências como organização do desenvolvimento, facilidade de manutenção e adequação ao problema. Metodologias e ferramentas de desenvolvimento estão em constante evolução, exigindo compreensão de seus conceitos.

19. Categorias de Linguagens

Compare duas alternativas hipotéticas sob a perspectiva de “Categorias de Linguagens”. Indique em que cenário cada alternativa faria mais sentido.

Uma alternativa pode ser uma linguagem voltada para um domínio específico, fazendo mais sentido quando o projeto possui necessidades bem definidas. Outra pode ser uma linguagem de propósito mais geral, sendo mais adequada quando o sistema envolve diferentes tipos de problemas e exige maior flexibilidade. A escolha deve considerar o domínio da aplicação.

20. Trade-off no projeto de linguagem

Você precisa defender uma decisão profissional relacionada a “Trade-off no projeto de linguagem”. Estructure uma justificativa curta com contexto, critério e trade-off.

Em um sistema que exige alta confiabilidade, por exemplo, eu priorizaria uma linguagem com mecanismos de verificação de tipos e tratamento de exceções. O critério seria a confiabilidade, pois erros precisam ser detectados e tratados adequadamente. O trade-off é aceitar possíveis custos ou restrições adicionais em troca de maior segurança e confiabilidade.

21. Métodos de Implementação

Caso prático: uma equipe precisa escolher uma linguagem para um novo sistema. Mostre como “Métodos de Implementação” deve entrar na análise e indique uma consequência concreta da decisão.

A equipe deve analisar como a linguagem será implementada, considerando alternativas como compilação, interpretação ou sistemas híbridos. O critério deve ser a necessidade de desempenho, custo e ambiente de execução. Uma consequência concreta é que uma implementação mais otimizada pode aumentar o tempo ou custo de compilação, mas produzir código mais rápido na execução.

22. Compilação

Procure o erro: alguém afirmou que não é preciso considerar “Compilação” quando o código já funciona. Identifique o problema desse raciocínio e reescreva a conclusão.

O erro está em pensar que, porque o programa funciona, o processo de compilação deixa de ser relevante. A compilação influencia custos, otimização e desempenho do programa. Portanto, mesmo com código funcional, deve-se considerar o custo de compilação e o nível de otimização adequado ao ambiente de execução.

23. Interpretação pura

Sugira uma abordagem para avaliar “Interpretação pura” durante uma revisão técnica. Apresente o critério usado e uma evidência que sustentaria sua recomendação.

Na revisão técnica, deve-se avaliar a interpretação pura principalmente pelo critério de custo e desempenho de execução. Ela pode ser adequada quando a facilidade de desenvolvimento e execução sem uma etapa de compilação prévia for mais importante. A evidência para recomendá-la seria a necessidade de executar o programa repetidamente sem priorizar tanto a velocidade de execução.

24. Sistemas de Implementação Híbridos

Compare duas alternativas hipotéticas sob a perspectiva de “Sistemas de Implementação Híbridos”. Indique em que cenário cada alternativa faria mais sentido.

Uma alternativa híbrida pode combinar características de compilação e interpretação, fazendo sentido quando o sistema precisa equilibrar portabilidade, flexibilidade e desempenho. Uma alternativa baseada apenas em compilação faria mais sentido quando o desempenho de execução for prioridade. Assim, a escolha depende do equilíbrio desejado entre velocidade, flexibilidade e custos de implementação. Diferentes estratégias de implementação envolvem diferentes custos de compilação e execução.

25. Sistemas de Implementação Just in Time

Você precisa defender uma decisão profissional relacionada a “Sistemas de Implementação Just in Time”. Estruture uma justificativa curta com contexto, critério e trade-off.

Para um sistema que precisa executar código com frequência e ainda manter flexibilidade, eu consideraria uma implementação Just in Time. O critério seria equilibrar desempenho e custo de compilação, aproveitando a possibilidade de otimizar o código durante a execução. O trade-off é aumentar o trabalho realizado durante a execução em troca de potencial melhoria de desempenho.

26. Pré-processadores

Caso prático: uma equipe precisa escolher uma linguagem para um novo sistema. Mostre como “Pré-processadores” deve entrar na análise e indique uma consequência concreta da decisão.

A equipe deve considerar os pré-processadores porque eles fazem parte da forma como o código pode ser preparado antes da compilação. O critério deve ser verificar se os recursos oferecidos são realmente necessários ao projeto e se facilitam seu desenvolvimento. Uma consequência concreta é que o pré-processamento pode permitir transformações ou inclusões no código antes da compilação, mas também acrescentar uma etapa ao processo de desenvolvimento.

27. Ambientes de Programação

Procure o erro: alguém afirmou que não é preciso considerar “Ambientes de Programação” quando o código já funciona. Identifique o problema desse raciocínio e reescreva a conclusão.

O erro está em considerar que um código funcional significa que o ambiente de programação não importa. O ambiente influencia custos de desenvolvimento, treinamento e escrita dos programas. Portanto, mesmo que o código funcione, deve-se avaliar se o ambiente facilita o desenvolvimento e reduz custos; um bom ambiente pode reduzir significativamente os custos de treinamento e escrita.